PROCESAMIENTO Y CONSULTA DE DATOS GEOESPACIALES CON DESPLIEGUE CONTINUO

Informe técnico · Big Data

SANTIAGO ARCILA GUTIÉRREZ
ALEJANDRO RESTREPO URIBE
NATALIA FLOREZ PÉREZ

INSTITUCIÓN UNIVERSITARIA DE ENVIGADO

Docente
ANDRÉS FELIPE HERNÁNDEZ MARULANDA

Envigado, 10 de octubre de 2026

Repositorio: github.com/santiarcila23/bigdata-geoespacial

Resumen

El sistema descarga 1,46 millones de delitos de Chicago desde Kaggle, los limpia con Dask, los guarda como puntos GeoJSON en MongoDB, los agrega con Spark y los expone en una API Flask con un mapa web. Jenkins construye, prueba y despliega la API cada vez que cambia la rama principal del repositorio. Todo corre en contenedores y se levanta con un solo comando: `docker compose up -d --build`.

Resultados principales: 1.419.554 registros válidos (97,5 %), cargados en 110 s; 2.569 celdas de grilla y 28 zonas de alta concentración calculadas por Spark en 53 s; consultas por ubicación entre 95 ms y 6,5 s; tres builds de Jenkins exitosos, uno de ellos disparado por el merge de un pull request; y un benchmark en el que Dask fue unas 4 veces más rápido que Spark con este volumen de datos.

1. Objetivo

Construir un sistema completo y reproducible, no piezas sueltas: descarga automática, limpieza distribuida, almacenamiento geoespacial, procesamiento distribuido, consultas por ubicación y una API que se despliega sola cuando el código cambia, sin pasos manuales ocultos.

2. Datos

Se usó Crimes in Chicago de Kaggle (currie32/crimes-in-chicago), archivo `Chicago_Crimes_2012_to_2017.csv`. Cada fila es un delito reportado con fecha, tipo, descripción, lugar, arresto, distrito y coordenadas. Se cargó completo, sin muestreo.

Tabla 1. *Características del dataset*

Medida	Valor
Registros en el archivo	1.456.714
Tamaño	367 MB descomprimido (90 MB comprimido)
Periodo	enero de 2012 a enero de 2017
Registros válidos en MongoDB	1.419.554

El archivo trae solo 30 registros de 2017, por lo que el análisis anual compara 2012 a 2016.

3. Arquitectura

Ocho servicios corren en Docker Compose y se comunican por la red interna usando el nombre del servicio como host (`mongo`, `spark-master`, `dask-scheduler`, `api-staging`). Los datos persisten en volúmenes con nombre (`mongo_data`, `data`, `jenkins_home`), así que los contenedores se pueden recrear sin perder información.

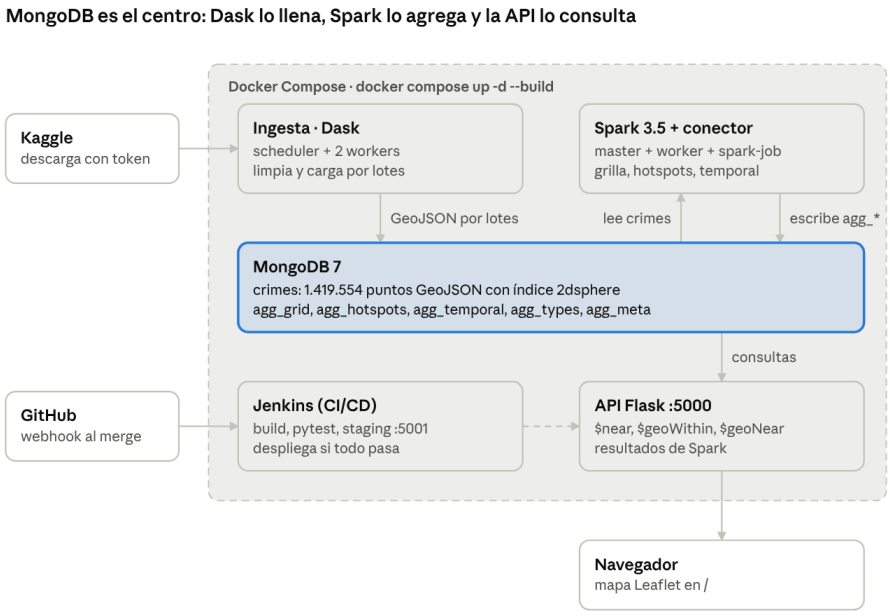


Figura 1. *Arquitectura del sistema: Dask llena MongoDB, Spark lo agrega y la API lo consulta; Jenkins solo despliega la API cuando sus pruebas pasan.*

Tabla 2. *Servicios del docker-compose.yml*

Servicio	Imagen	Puerto	Función
mongo	mongo:7.0	27018 (solo localhost)	Datos GeoJSON con índice 2dsphere
spark-master, spark-worker	apache/spark 3.5.3 + conector MongoDB 10.4	8081 (UI)	Clúster Spark standalone
spark-job	la de Spark	4040 mientras corre	Agregaciones (se ejecuta y termina)
dask-scheduler, dask-worker ×2	python 3.11 + Dask 2025.5	8787 (dashboard)	Clúster Dask
ingest	la de Dask	—	Kaggle → limpieza → MongoDB (se ejecuta y termina)
api	python 3.11 + Flask 3.1 + gunicorn	5000	API de producción y mapa
api-staging	la de la API	5001	Versión candidata que prueba Jenkins
jenkins	jenkins LTS + Docker CLI	8080	CI/CD configurado como código

4. Decisiones técnicas

- **Una sola imagen para todo Dask.** Cliente de ingesta, scheduler y workers usan la misma imagen, porque Dask exige versiones idénticas de Python y librerías en todos los procesos.

- **Conector de Spark resuelto al construir la imagen.** El MongoDB Spark Connector 10.4 y sus dependencias se descargan en el `docker build` y quedan en `/opt/spark/jars`; los trabajos no dependen de internet.
- **Driver de Spark en su propio contenedor.** `spark-job` envía en modo client y anuncia su IP interna (`spark.driver.host`) para que el executor se conecte de vuelta.
- **Idempotencia.** El `_id` de cada documento es el ID original y la carga usa `insert_many(ordered=False)`, así que re-ejecutar no duplica. La ingesta se salta si ya hay datos y Spark no recalcula si sus resultados corresponden a la última ingesta.
- **Credenciales fuera del repositorio.** El token de Kaggle y la clave de Jenkins viven en `.env` (en `.gitignore`) y en la credencial `kaggle-api` de Jenkins, que el pipeline inyecta con `withCredentials`. MongoDB solo se publica en `127.0.0.1`, puerto 27018, para no chocar con instalaciones locales.
- **Jenkins como código.** Con Configuration as Code y Job DSL arranca con el usuario, la credencial y el job ya creados; no hay asistente de instalación.
- **Despliegue con imagen candidata.** El pipeline construye `geo-api:candidate`, la prueba en staging (puerto 5001) y solo si todo pasa la retaguea como `geo-api:latest` y recrea la API de producción. Una prueba fallida deja producción intacta.
- **Archivos comprimidos detectados por contenido.** Kaggle entregó el CSV comprimido pero con extensión `.csv`; la ingesta revisa la firma ZIP del archivo, no la extensión.

5. Ingesta y limpieza con Dask

El 97,5 % de los registros pasó la limpieza: 1.419.554 de 1.456.714, en 109,8 s de punta a punta (descarga 26,4 s, limpieza 41,7 s, carga 40,7 s). La descarga la hace el propio pipeline con la CLI de Kaggle. El CSV se lee con `dask.dataframe.read_csv` en 5 particiones de 64 MB repartidas entre 2 workers, con todas las columnas como texto para evitar errores de inferencia de tipos entre particiones.

Tabla 3. *Reglas de limpieza y registros descartados*

Regla	Justificación	Descartados
Latitud o longitud nula o no numérica	Sin coordenadas no se puede ubicar ni indexar	37.083 (2,5 %)
Fuera de rango (lat ±90, lon ±180)	MongoDB rechaza puntos inválidos en un índice 2dsphere	0
Fuera del área de estudio (lon −87,96 a −87,50; lat 41,62 a 42,05)	Puntos fuera de Chicago son errores de georreferenciación y distorsionan la grilla	77
Fecha no interpretable	Necesaria para el análisis temporal	0
ID faltante	El ID es la clave primaria (`_id`)	0
ID duplicado	Contaría dos veces el mismo delito	0
Total válido		**1.419.554 (97,5 %)**

Cada registro válido se convierte en un punto GeoJSON `{"type": "Point", "coordinates": [lon, lat]}` (GeoJSON usa el orden longitud, latitud). La fecha se guarda como `Date` de BSON, arresto y doméstico como booleanos, y distrito y área comunitaria como enteros. El resultado

limpio se guarda en Parquet, que reutiliza el benchmark, y cada worker inserta su partición en MongoDB con `insert_many(ordered=False)` en lotes de 5.000.

Índices creados: `location_2dsphere` para las consultas espaciales, `date_1` para filtros por fecha y `category_date` para filtros por tipo y fecha.

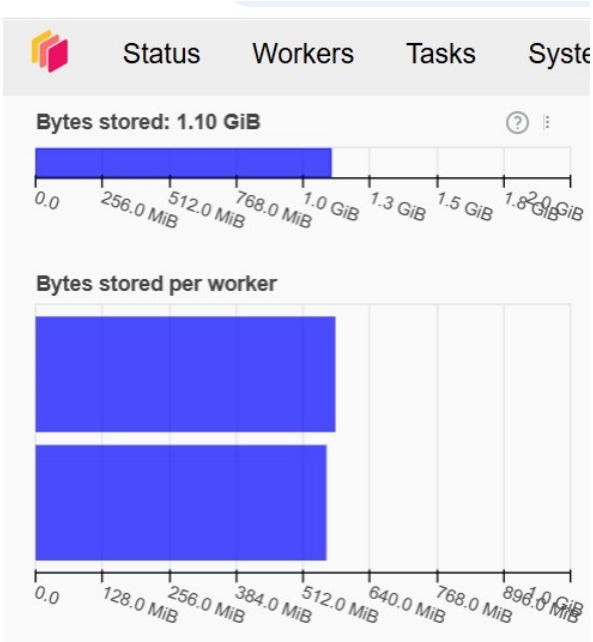


Figura 2. *Dashboard de Dask al terminar la carga: los dos workers guardan 1,10 GiB.*

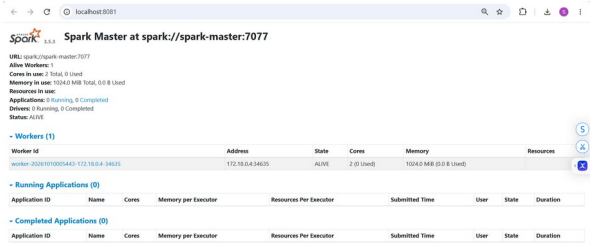


Figura 3. *Spark Master con el worker activo (2 cores, 1024 MiB).*

6. Procesamiento distribuido con Spark

Spark leyó los 1.419.554 puntos desde MongoDB con el conector, en 7 particiones, y escribió 6 colecciones nuevas en 53,2 s con un executor de 2 núcleos y 900 MiB: `agg_grid`, `agg_hotspots`, `agg_temporal`, `agg_category_hour`, `agg_types` y `agg_meta`.

Grilla espacial

Cada punto se asigna a la celda `(floor(lat / 0,005), floor(lon / 0,005))`, de unos 550 × 410 m (0,23 km²). Hay 2.569 celdas con datos, con un promedio de 552,6 delitos por celda y una desviación de 633. Por celda se guarda el conteo, el polígono GeoJSON, el centro, la densidad por km², la tasa de arresto y la categoría más frecuente.

Zonas de alta concentración

El percentil 99 del conteo por celda es 2.768 delitos. Lo superan 28 celdas, todas en el centro (The Loop y Near North Side) y en el oeste (Austin, Garfield Park, Humboldt Park).

Tabla 4. *Las tres celdas con más delitos*

Rank	Centro (lat, lon)	Zona	Delitos	z-score	Densidad (por km²)	Categoría principal	Arresto
1	41,8825, −87,6275	The Loop	10.105	15,1	43.810	THEFT (6.669)	36,9 %
2	41,8775, −87,6275	The Loop (sur)	5.451	7,7	23.631	THEFT (2.525)	30,6 %
3	41,8975, −87,6225	Near North Side	5.247	7,4	22.754	THEFT (3.614)	35,7 %

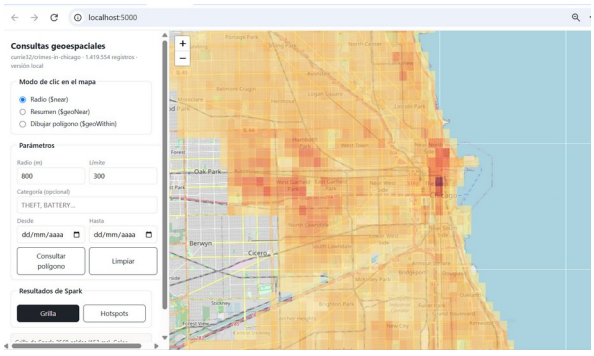


Figura 4. *Grilla de Spark en el mapa: la mayor concentración está en The Loop y en West Garfield Park.*

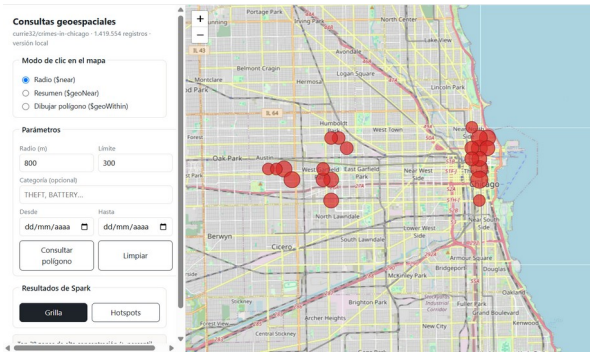


Figura 5. *Las 28 zonas de alta concentración (percentil 99).*

Comportamiento temporal

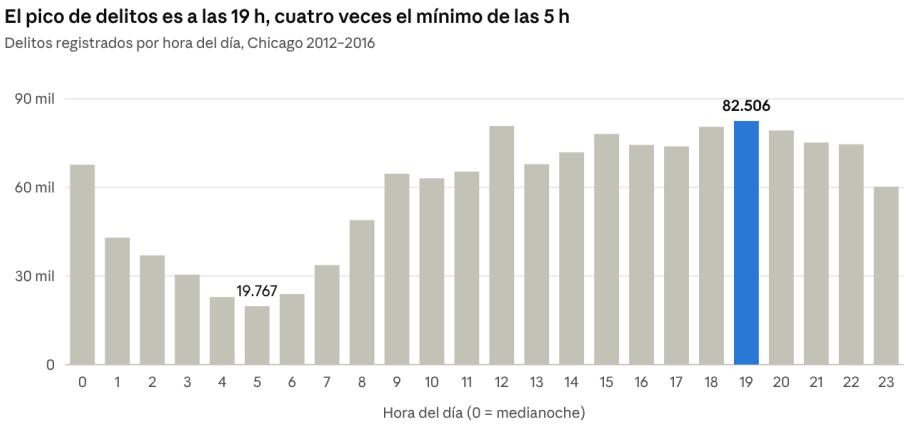


Figura 6. *Delitos por hora del día, colección agg_temporal calculada por Spark.*

El viernes es el día con más delitos (212.947; 15,0 %) y el domingo el de menos (196.490; 13,8 %). Julio es el mes más alto (133.788) y febrero el más bajo (96.097). Entre 2012 y 2016 los delitos registrados bajaron un 24,9 %, de 334.731 a 251.273 por año.

Por categoría

Las seis categorías más frecuentes suman el 73 % de los delitos.

Tabla 5. *Delitos por categoría*

Categoría	Delitos	Participación	Tasa de arresto
THEFT	322.423	22,7 %	11,3 %
BATTERY	258.929	18,2 %	23,2 %
CRIMINAL DAMAGE	152.811	10,8 %	6,6 %
NARCOTICS	131.173	9,2 %	99,3 %
ASSAULT	89.507	6,3 %	23,7 %
OTHER OFFENSE	85.356	6,0 %	21,5 %

NARCOTICS tiene 99,3 % de arresto porque estos delitos casi siempre se registran cuando la policía detiene a alguien; el robo (THEFT) solo tiene 11,3 %.

Verificación

La suma de los conteos de `agg_grid` es 1.419.554, igual al número de documentos de `crimes`. Además, la misma agregación calculada por separado con Dask en el benchmark (sección 9) dio las mismas 2.569 celdas.

7. Consultas geoespaciales y API

Las tres consultas exigidas funcionan con parámetros variables y responden entre 95 ms y 6,5 s sobre 1,42 millones de puntos. Todas validan sus parámetros (latitud y longitud en rango, radio entre 1 m y 50 km, polígonos cerrados) y aceptan filtros por categoría, rango de fechas, año y arresto.

Por radio con $near. `GET /api/crimes/near?lat=&lon=&radius=` devuelve los registros del más cercano al más lejano:

```
db.crimes.find({ location: { $near: {
  $geometry: { type: "Point", coordinates: [lon, lat] },
  $maxDistance: radius } } }).limit(limit)
```

Dentro de un polígono con $geoWithin. `POST /api/crimes/within` recibe un Polygon, MultiPolygon o Feature GeoJSON:

```
db.crimes.find({ location: { $geoWithin: { $geometry: <Polygon GeoJSON> } } })
```

Agregación con $geoNear. `GET /api/crimes/summary` resume por categoría lo que hay alrededor de un punto, con la distancia promedio y la mínima:

```
db.crimes.aggregate([
  { $geoNear: { near: { type: "Point", coordinates: [lon, lat] },
                distanceField: "distance_m",
                maxDistance: radius, spherical: true } },
  { $group: { _id: "$category", count: { $sum: 1 },
              avg_distance_m: { $avg: "$distance_m" } } },
  { $sort: { count: -1 } }
])
```

Tabla 6. *Tiempos de respuesta medidos (cabecera X-Response-Time-ms)*

Consulta	Parámetros	Registros	Tiempo
$near	The Loop, radio 1 km, límite 200	200	95 ms
$geoNear	West Garfield Park, radio 800 m	3.173	135 ms
$geoWithin	Triángulo sobre Garfield Park	41.956	931 ms
$geoWithin	Hexágono sobre Humboldt y Garfield Park	77.015	2.721 ms
$geoNear	The Loop, radio 1 km	zona más densa	6.459 ms

El tiempo crece con la cantidad de puntos que caen en el área: `$near` con límite se detiene en los primeros 200, mientras que `$geoWithin` con conteo y `$geoNear` agrupado recorren todos los puntos del área. The Loop es la zona de mayor densidad (sección 6), por eso el resumen a 1 km tarda más.

La API expone además los resultados de Spark en `/api/stats/grid`, `/hotspots`, `/temporal`, `/types` y `/meta`, y un mapa Leaflet en `/` para consultar con clics, dibujar polígonos y ver la grilla. Un polígono con lados cruzados devuelve HTTP 400 con un mensaje claro en lugar de un error interno.

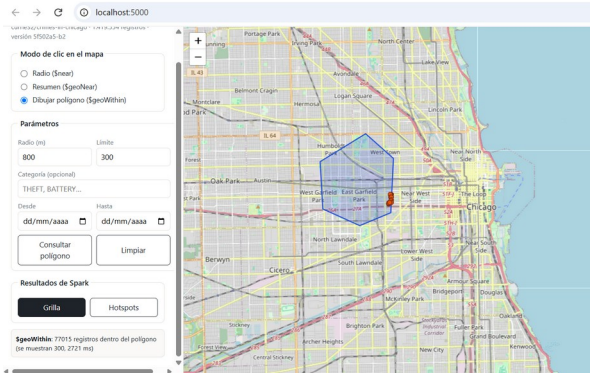


Figura 7. *Consulta $geoWithin con un hexágono: 77.015 registros dentro.*

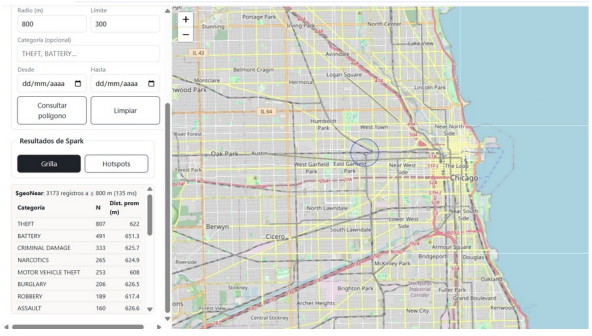


Figura 8. *Agregación $geoNear: 3.173 delitos a 800 m, por categoría.*

8. Integración y despliegue continuo

Un merge a `main` llega a producción en unos 3 minutos sin intervención manual, y solo si pasan las pruebas. El `Jenkinsfile` del repositorio define nueve etapas: checkout, build de imágenes, pruebas unitarias con pytest, levantar servicios, ingesta (solo si MongoDB está vacío o se pide `FORCE_INGEST`), Spark (solo si hubo ingesta nueva o se pide `FORCE_SPARK`), staging de la imagen candidata, pruebas básicas contra la API y despliegue. El job se dispara con el webhook de GitHub y, como respaldo, revisando el repositorio cada 2 minutos.

Las pruebas son de dos tipos. Las **unitarias** (26) validan parámetros, construcción de consultas y endpoints con una base de datos simulada. Las **básicas contra la API** (9) corren contra la candidata en staging con los datos reales: salud, `$near`, `$geoWithin`, `$geoNear`, los cuatro resultados de Spark y el manejo de parámetros inválidos. Si cualquiera falla, la etapa de despliegue no se ejecuta y producción sigue con la imagen anterior.

Tabla 7. *Builds del pipeline*

Build	Commit	Origen	Duración	Pruebas	Resultado	Versión desplegada
#3	3facdbf	Push de los resultados del benchmark	2 min 29 s	35 (26 + 9)	Exitoso	3facdbf-b3
#2	5f502a5	Merge del pull request fix/poligono-invalido	3 min 2 s	35 (26 + 9)	Exitoso	5f502a5-b2
#1	f3f44b4	Primera revisión del repositorio	4 min 9 s	34 (25 + 9)	Exitoso	f3f44b4-b1

El build #2 muestra el flujo completo: rama de trabajo, pull request, merge, detección automática del commit, pruebas y despliegue. La versión desplegada se verifica en `/health`, que pasó de `f3f44b4-b1` a `5f502a5-b2` y luego a `3facdbf-b3`. El build de imágenes bajó de 2 min 10 s a 5 s porque Docker reutiliza las capas que no cambiaron.

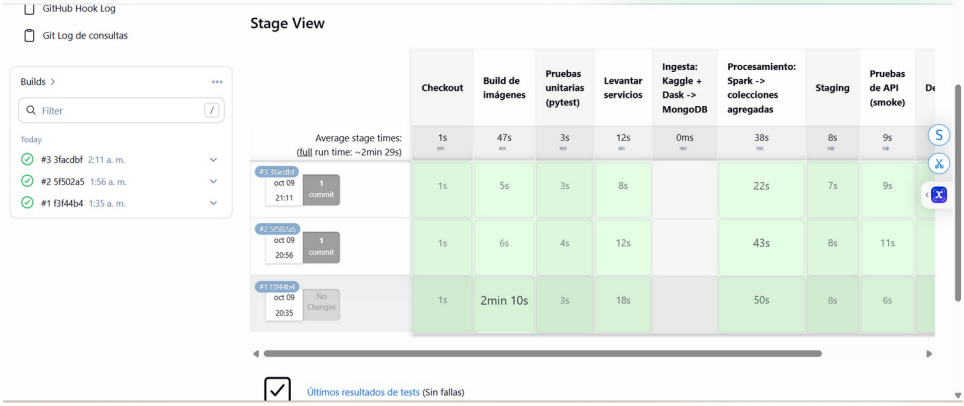


Figura 9. *Stage View de Jenkins con los tres builds en verde.*

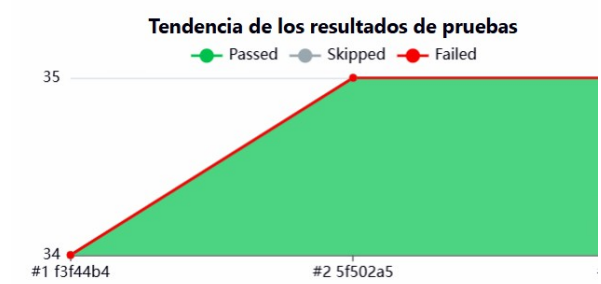


Figura 10. *Tendencia de pruebas: de 34 a 35 con la prueba nueva del polígono, sin fallas.*

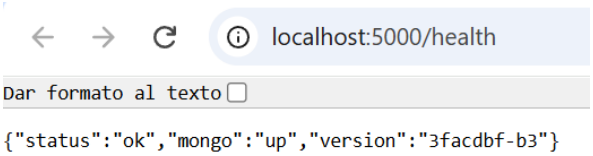


Figura 11. *Endpoint /health de producción con la versión del build #3.*

9. Comparación Dask frente a Spark

Con este volumen, Dask fue unas 4 veces más rápido que Spark en caliente y usó menos memoria por worker. Duplicar los núcleos casi no mejoró a ninguno de los dos.

Operación medida: la agregación por grilla (conteo por celda de 0,005°) sobre los 1.419.554 registros limpios en Parquet, con 3 repeticiones por configuración. Para comparar con los mismos

núcleos, Spark usó workers de 2 cores y Dask workers de 1 hilo. Los dos motores dieron el mismo resultado en todas las ejecuciones: 2.569 celdas y 1.419.554 filas.

Tabla 8. *Resultados del benchmark*

Motor	Configuración	Núcleos	1.ª ejecución (s)	Mediana (s)	Mínimo (s)	Memoria pico (MB)
Spark	1 worker × 2 cores	2	10,84	1,52	1,00	561
Spark	2 workers × 2 cores	4	15,22	1,73	1,34	948
Dask	2 workers × 1 hilo	2	2,41	0,34	0,34	514
Dask	4 workers × 1 hilo	4	1,20	0,37	0,37	675

La memoria de Dask es el RSS máximo de los procesos worker (unos 260 MB cada uno). La de Spark es el RSS de los procesos executor según la API REST del driver (unos 470 MB cada uno). En Spark la cifra incluye la JVM completa, así que buena parte es un consumo fijo que no depende de los datos y que además creció entre repeticiones (de 424 a 561 MB con un executor).

Análisis

1. **El costo fijo domina.** La operación solo lee dos columnas de 1,4 millones de filas, unos 23 MB. Con datos tan pequeños pesan más el arranque y la coordinación que el cálculo, y Spark tiene un costo fijo mayor: JVM, planificación del trabajo y envío de tareas a los executors.
2. **Más workers no ayudó.** En Spark, pasar a 2 workers empeoró el arranque en frío (de 10,8 a 15,2 s, porque hay que lanzar dos executors) y no mejoró la mediana. En Dask el arranque en frío bajó a la mitad (de 2,4 a 1,2 s), pero en caliente quedó igual (0,34 contra 0,37 s): repartir 23 MB entre más procesos cuesta lo mismo que lo que se gana.
3. **La primera ejecución es mucho más lenta en ambos,** sobre todo en Spark (10,8 s contra 1,0 s en la tercera): carga de clases, compilación del plan y primera lectura del Parquet desde disco.
4. **Todo corrió en un solo equipo** (12 CPU, 6 GB para Docker). Los workers comparten disco y memoria, así que no se midió lo que Spark hace mejor: repartir datos que no caben en una sola máquina y recuperarse si un nodo falla.

Cuándo conviene cada uno

Según estas mediciones, **Dask** conviene cuando los datos caben en uno o pocos equipos y el equipo trabaja en Python: arranca rápido, usa menos memoria y reutiliza lo que se sabe de pandas; por eso se usó para la ingesta. **Spark** conviene cuando el volumen crece varios órdenes de magnitud, hay un clúster de varios nodos, se necesita tolerancia a fallos o se aprovechan conectores como el de MongoDB, que aquí permitió leer directamente desde la base sin pasos intermedios.

Los resultados completos están en `benchmark/results.md` del repositorio.

10. Conclusiones

- El sistema cumple el ciclo completo con un solo comando: descarga 1,46 millones de registros de Kaggle, deja 1,42 millones válidos en MongoDB y los procesa con Spark en menos de 3 minutos en total.
- La concentración del delito es extrema: la celda más cargada de The Loop reúne 10.105 delitos, 18 veces el promedio por celda, y las 28 celdas del percentil 99 están en el centro y en el oeste de la ciudad.
- El índice 2dsphere hace que las consultas espaciales respondan en milisegundos cuando el área es acotada; el costo crece con la cantidad de puntos del área, no con el tamaño total de la colección.
- Con este volumen, Dask fue más rápido y liviano que Spark, y agregar workers no ayudó: el costo fijo de coordinación pesa más que el cálculo. Spark se justifica con más volumen, varios nodos o conectores como el de MongoDB.
- Lo más difícil fue la integración: el archivo de Kaggle llegaba comprimido con extensión `.csv`, el puerto 27017 estaba ocupado por una instalación local de MongoDB y Jenkins debía usar Docker desde un contenedor. Cada problema quedó resuelto en el código para que el sistema se levante igual en otro equipo.

Referencias

Apache Software Foundation. (2024). Spark 3.5.3 documentation. https://spark.apache.org/docs/3.5.3/

Currie32. (2017). Crimes in Chicago [Conjunto de datos]. Kaggle. https://www.kaggle.com/datasets/currie32/crimes-in-chicago

Dask Development Team. (2025). Dask DataFrame. https://docs.dask.org/en/stable/dataframe.html

Jenkins Project. (2026). Configuration as Code. https://www.jenkins.io/projects/jcasc/

MongoDB, Inc. (2026). Geospatial queries. https://www.mongodb.com/docs/manual/geospatial-queries/

MongoDB, Inc. (2026). MongoDB Spark Connector. https://www.mongodb.com/docs/spark-connector/current/

Parte del código se desarrolló con apoyo de un asistente de IA (Claude, de Anthropic) y fue revisado, probado y adaptado por el equipo.